DS-604 Project Documentation

Formula 1 World Championship (1950–2024): Database Design and Performance Analysis

Milestone 2: ER Diagram, Relational Schema Diagram and DDL Scripts

Student ID:

202618052

202618016

202618013

Contents

No.	Name	Page No.
1.	Overview	3
2.	Entity-Relationship (ER) Diagram	4
3.	Relational Schema	9
4.	DDL Scripts	10

1. Overview

This document is the second milestone deliverable for the DS604 group project. Building on the dataset description and analytical queries submitted in Milestone 1, it presents the database design for the Formula 1 World Championship dataset: the Entity-Relationship (ER) diagram, the relational schema diagram and the DDL scripts that create the tables.

Dataset Scope

Dataset Name: Formula 1 World Championship 1950–2024 (Kaggle)

Dataset Link: https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

2. Entity-Relationship (ER) Diagram

The conceptual model contains 12 entities and 20 one-to-many relationships. RACE is the central entity: every race is held at one CIRCUIT, and all race-level records (results, sprint results, qualifying, lap times, pit stops and championship standings) are linked to a race and to the DRIVER and/or CONSTRUCTOR involved. RESULT and SPRINT_RESULT are also linked to a STATUS that describes how the driver finished. The diagram is drawn in Chen notation; the symbols used are listed below.

2.1 Notation

Symbol	Meaning
Rectangle	Strong entity (has its own key attribute)
Double rectangle	Weak entity (identified through its owner entities, race and driver, plus a partial key)
Diamond	Relationship
Double diamond	Identifying relationship (links a weak entity to an owner entity)
Oval with underlined name	Key attribute (dashed underline = partial key of a weak entity)
Single line / double line	Partial participation / total participation (foreign key is NOT NULL)
1 and N	Cardinality at each end of the relationship (one-to-many)

The diagram shows every attribute of the reduced schema. Foreign keys are not drawn as attributes; they appear as relationships and are added to the tables when the diagram is mapped to relations (Section 3.3). The diagram is given on the next page.

2.2 ER Diagram

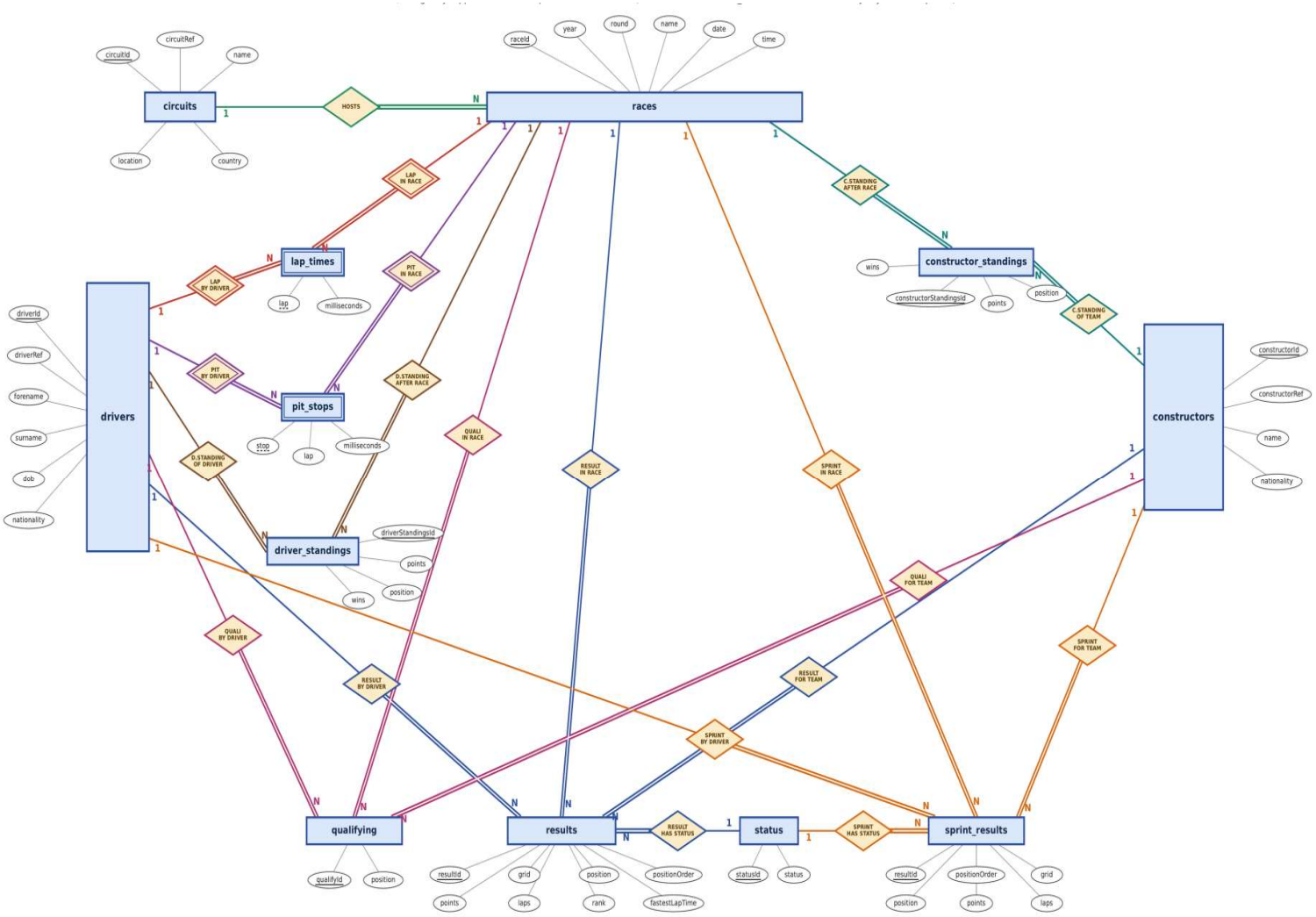


Figure 1: ER diagram of the reduced Formula 1 database

2.3 Entities and Attributes

Entity	Type	Key Attribute(s)	Attributes shown in the diagram
CIRCUIT	Strong	circuitId	circuitId, circuitRef, name, location, country
RACE	Strong	raceId	raceId, year, round, name, date, time
DRIVER	Strong	driverId	driverId, driverRef, forename, surname, dob, nationality
CONSTRUCTOR	Strong	constructorId	constructorId, constructorRef, name, nationality
STATUS	Strong	statusId	statusId, status
RESULT	Strong	resultId	resultId, points, position, grid, laps, rank, positionOrder, fastestLapTime
SPRINT_RESULT	Strong	resultId	resultId, points, position, positionOrder, grid, laps
QUALIFYING	Strong	qualifyId	qualifyId, position
LAP_TIME	Weak	raceId, driverId, lap (partial key: lap)	lap, milliseconds
PIT_STOP	Weak	raceId, driverId, stop (partial key: stop)	stop, lap, milliseconds
DRIVER_STANDING	Strong	driverStandingsId	driverStandingsId, points, position, wins
CONSTRUCTOR_STANDING	Strong	constructorStandingsId	constructorStandingsId, wins, points, position

LAP_TIME and PIT_STOP are weak entities: a lap number or a stop number is only unique within a particular driver's race, so each is identified by its owners (RACE and DRIVER) together with its partial key (lap or stop). All other entities have their own identifier in the dataset.

3. Relational Schema

3.1 Relational Schema Diagram

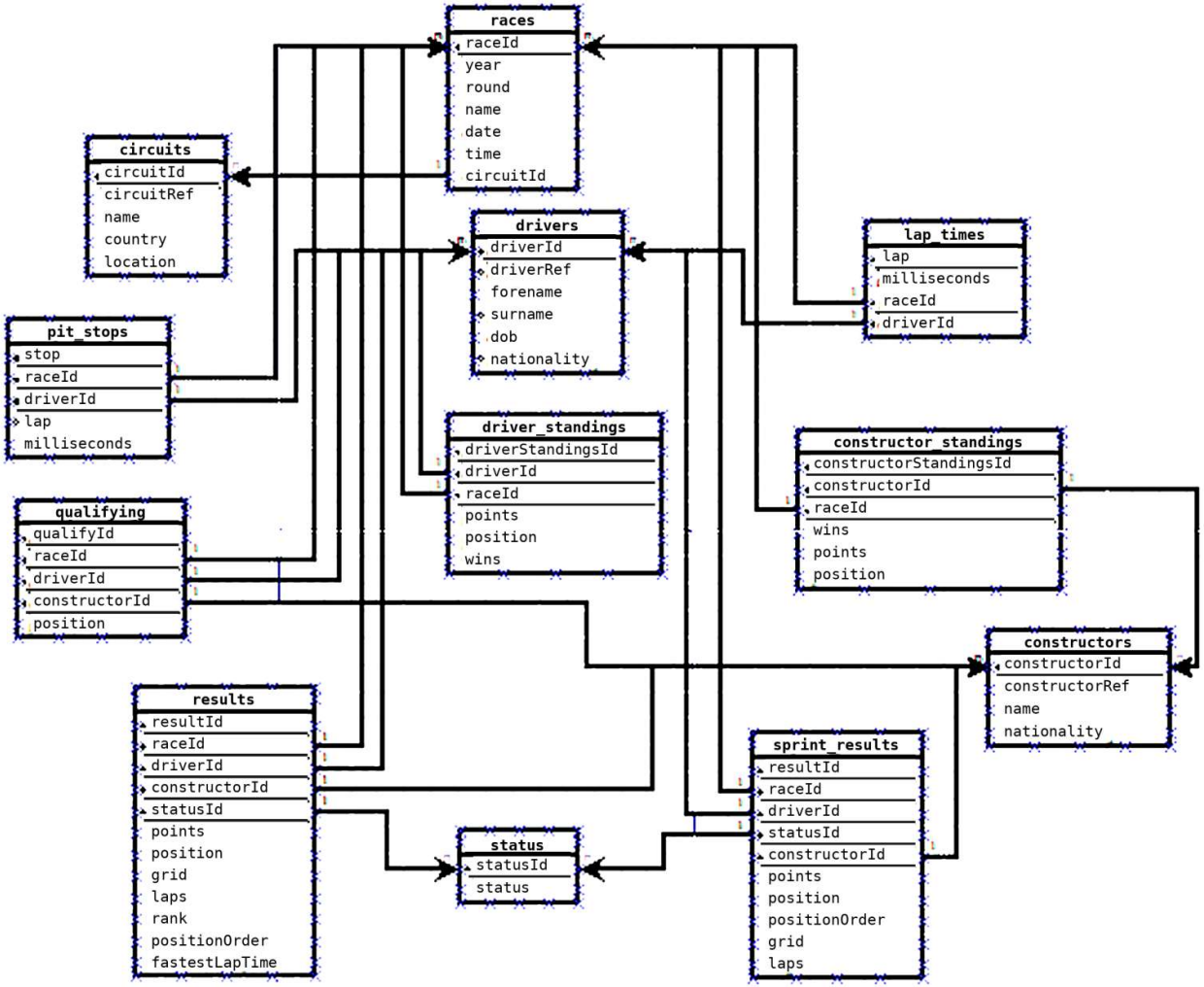


Figure 2: Relational schema diagram of the 12 tables. Underlined attributes are primary and foreign keys; each line connects a foreign key to the primary key it reference

4. DDL Scripts

The script below creates all 12 tables in PostgreSQL. Tables are created parent-first so that every FOREIGN KEY target exists (see Section 4).

```
-- Clean reset
DROP TABLE IF EXISTS constructor_standings, driver_standings, pit_stops,
    lap_times, sprint_results, qualifying, results, races,
    status, drivers, constructors, circuits CASCADE;

-- 1. Independent (parent) tables

CREATE TABLE circuits (
    circuitId   INT           PRIMARY KEY,
    circuitRef  VARCHAR(50)   UNIQUE NOT NULL,
    name        VARCHAR(100)  NOT NULL,
    location    VARCHAR(100),
    country     VARCHAR(60)
);

CREATE TABLE constructors (
    constructorId   INT          PRIMARY KEY,
    constructorRef  VARCHAR(50)  UNIQUE NOT NULL,
    name            VARCHAR(100) NOT NULL,
    nationality     VARCHAR(50)
);

CREATE TABLE drivers (
    driverId    INT          PRIMARY KEY,
    driverRef   VARCHAR(50)  UNIQUE NOT NULL,
    forename    VARCHAR(50)  NOT NULL,
    surname     VARCHAR(50)  NOT NULL,
    dob         DATE,
    nationality VARCHAR(50)
);

CREATE TABLE status (
    statusId  INT          PRIMARY KEY,
    status    VARCHAR(100) NOT NULL
);

-- 2. races (depends on circuits)

CREATE TABLE races (
    raceId     INT          PRIMARY KEY,
    year       SMALLINT     NOT NULL,
    round      SMALLINT      NOT NULL,
    name       VARCHAR(100) NOT NULL,
    date       DATE         NOT NULL,
    time       TIME         NULL,          -- not recorded for early seasons
    circuitId  INT          NOT NULL,

    CONSTRAINT uq_races_year_rnd UNIQUE (year, round),
    CONSTRAINT fk_races_circuit  FOREIGN KEY (circuitId)
        REFERENCES circuits (circuitId)
        ON UPDATE CASCADE ON DELETE RESTRICT,
```

```
    CONSTRAINT ck_races_year     CHECK (year BETWEEN 1950 AND 2100),
    CONSTRAINT ck_races_round    CHECK (round > 0)
);

-- 3. Race-level fact tables

CREATE TABLE results (
    resultId        INT           PRIMARY KEY,
    raceId          INT           NOT NULL,
    driverId        INT           NOT NULL,
    constructorId   INT           NOT NULL,
    statusId        INT           NOT NULL,
    points          DECIMAL(5,2)  NOT NULL DEFAULT 0,
    position        SMALLINT      NULL,    -- NULL when not classified (DNF/DSQ)
    grid            SMALLINT      NOT NULL DEFAULT 0,  -- 0 = pit lane start
    laps            SMALLINT      NOT NULL DEFAULT 0,
    "rank"          SMALLINT      NULL,    -- fastest-lap rank (1 = fastest lap of
race)
    positionOrder   SMALLINT      NOT NULL,
    fastestLapTime  VARCHAR(12)   NULL,    -- format m:ss.mmm

    CONSTRAINT uq_results_race_driver UNIQUE (raceId, driverId),
    CONSTRAINT fk_results_race        FOREIGN KEY (raceId)
        REFERENCES races (raceId)     ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_results_driver      FOREIGN KEY (driverId)
        REFERENCES drivers (driverId) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_results_constructor FOREIGN KEY (constructorId)
        REFERENCES constructors (constructorId) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_results_status      FOREIGN KEY (statusId)
        REFERENCES status (statusId)  ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_results_points CHECK (points >= 0),
    CONSTRAINT ck_results_grid   CHECK (grid >= 0),
    CONSTRAINT ck_results_laps   CHECK (laps >= 0)
);

CREATE TABLE qualifying (
    qualifyId      INT      PRIMARY KEY,
    raceId         INT      NOT NULL,
    driverId       INT      NOT NULL,
    constructorId  INT      NOT NULL,
    position       SMALLINT NOT NULL,

    CONSTRAINT uq_qualifying_race_driver UNIQUE (raceId, driverId),
    CONSTRAINT fk_qualifying_race        FOREIGN KEY (raceId)
        REFERENCES races (raceId)        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_qualifying_driver      FOREIGN KEY (driverId)
        REFERENCES drivers (driverId)    ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_qualifying_constructor FOREIGN KEY (constructorId)
        REFERENCES constructors (constructorId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_qualifying_pos CHECK (position IS NULL OR position > 0)
);

CREATE TABLE sprint_results (
    resultId       INT           PRIMARY KEY,
    raceId         INT           NOT NULL,
    driverId       INT           NOT NULL,
```

```
    statusId       INT           NOT NULL,
    constructorId  INT           NOT NULL,
    points         DECIMAL(5,2)  NOT NULL DEFAULT 0,
    position       SMALLINT      NULL,
    positionOrder  SMALLINT      NOT NULL,
    grid           SMALLINT      NOT NULL DEFAULT 0,
    laps           SMALLINT      NOT NULL DEFAULT 0,

    CONSTRAINT uq_sprint_race_driver     UNIQUE (raceId, driverId),
    CONSTRAINT fk_sprint_race            FOREIGN KEY (raceId)
        REFERENCES races (raceId)        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_sprint_driver          FOREIGN KEY (driverId)
        REFERENCES drivers (driverId)    ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_sprint_status          FOREIGN KEY (statusId)
        REFERENCES status (statusId)     ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_sprint_constructor     FOREIGN KEY (constructorId)
        REFERENCES constructors (constructorId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_sprint_points CHECK (points >= 0)
);

-- 4. Lap / pit-stop tables (composite primary keys)

CREATE TABLE lap_times (
    raceId        INT      NOT NULL,
    driverId      INT      NOT NULL,
    lap           SMALLINT NOT NULL,
    milliseconds  INT      NOT NULL,

    CONSTRAINT pk_lap_times        PRIMARY KEY (raceId, driverId, lap),
    CONSTRAINT fk_laptimes_race    FOREIGN KEY (raceId)
        REFERENCES races (raceId)  ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_laptimes_driver  FOREIGN KEY (driverId)
        REFERENCES drivers (driverId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_laptimes_lap CHECK (lap > 0),
    CONSTRAINT ck_laptimes_ms  CHECK (milliseconds > 0)
);

CREATE TABLE pit_stops (
    raceId        INT      NOT NULL,
    driverId      INT      NOT NULL,
    stop          SMALLINT NOT NULL,
    lap           SMALLINT NOT NULL,
    milliseconds  INT      NOT NULL,

    CONSTRAINT pk_pit_stops        PRIMARY KEY (raceId, driverId, stop),
    CONSTRAINT fk_pitstops_race    FOREIGN KEY (raceId)
        REFERENCES races (raceId)  ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_pitstops_driver  FOREIGN KEY (driverId)
        REFERENCES drivers (driverId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_pitstops_stop CHECK (stop > 0),
    CONSTRAINT ck_pitstops_lap  CHECK (lap > 0),
    CONSTRAINT ck_pitstops_ms   CHECK (milliseconds > 0)
);

-- 5. Championship standings
```

```
CREATE TABLE driver_standings (
    driverStandingsId  INT           PRIMARY KEY,
    raceId             INT           NOT NULL,
    driverId           INT           NOT NULL,
    points             DECIMAL(6,2)  NOT NULL DEFAULT 0,
    position           SMALLINT      NOT NULL,
    wins               SMALLINT      NOT NULL DEFAULT 0,

    CONSTRAINT uq_dstand_race_driver UNIQUE (raceId, driverId),
    CONSTRAINT fk_dstand_race        FOREIGN KEY (raceId)
        REFERENCES races (raceId)    ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_dstand_driver      FOREIGN KEY (driverId)
        REFERENCES drivers (driverId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_dstand_points CHECK (points >= 0),
    CONSTRAINT ck_dstand_wins   CHECK (wins >= 0)
);

CREATE TABLE constructor_standings (
    constructorStandingsId  INT           PRIMARY KEY,
    raceId                  INT           NOT NULL,
    constructorId           INT           NOT NULL,
    points                  DECIMAL(6,2)  NOT NULL DEFAULT 0,
    position                SMALLINT      NOT NULL,
    wins                    SMALLINT      NOT NULL DEFAULT 0,

    CONSTRAINT uq_cstand_race_constr    UNIQUE (raceId, constructorId),
    CONSTRAINT fk_cstand_race           FOREIGN KEY (raceId)
        REFERENCES races (raceId)       ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_cstand_constructor    FOREIGN KEY (constructorId)
        REFERENCES constructors (constructorId) ON UPDATE CASCADE ON DELETE RESTRICT,

    CONSTRAINT ck_cstand_points CHECK (points >= 0),
    CONSTRAINT ck_cstand_wins   CHECK (wins >= 0)
);

-- 6. Secondary indexes (speed up the 15 analytical queries' joins/filters)
--    Note: FK columns already get an index in InnoDB when no suitable
--    index exists; these cover the remaining access paths.

CREATE INDEX idx_races_year           ON races (year);
CREATE INDEX idx_races_circuit        ON races (circuitId);
CREATE INDEX idx_results_driver       ON results (driverId);
CREATE INDEX idx_results_constructor  ON results (constructorId);
CREATE INDEX idx_results_status       ON results (statusId);
CREATE INDEX idx_results_rank         ON results ("rank");
CREATE INDEX idx_qualifying_driver    ON qualifying (driverId);
CREATE INDEX idx_laptimes_driver      ON lap_times (driverId);
CREATE INDEX idx_pitstops_driver      ON pit_stops (driverId);
CREATE INDEX idx_dstand_driver        ON driver_standings (driverId);
CREATE INDEX idx_cstand_constructor   ON constructor_standings (constructorId);
CREATE INDEX idx_sprint_driver        ON sprint_results (driverId);
CREATE INDEX idx_drivers_nationality  ON drivers (nationality);
CREATE INDEX idx_circuits_country     ON circuits (country);
```